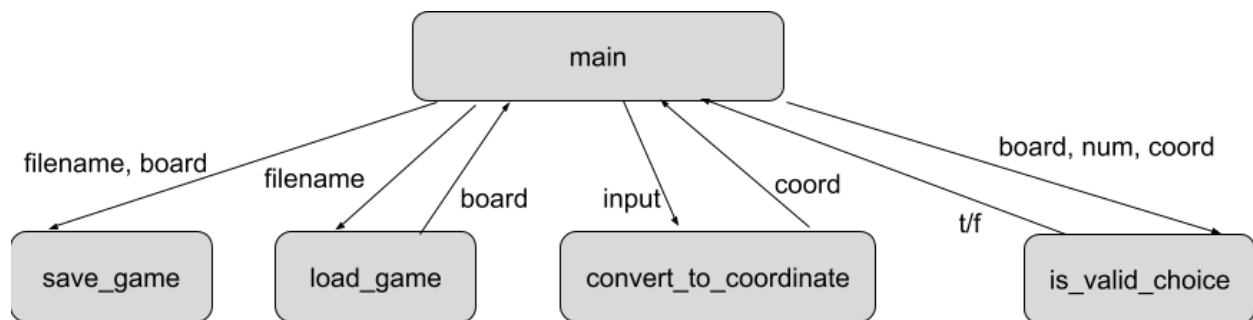
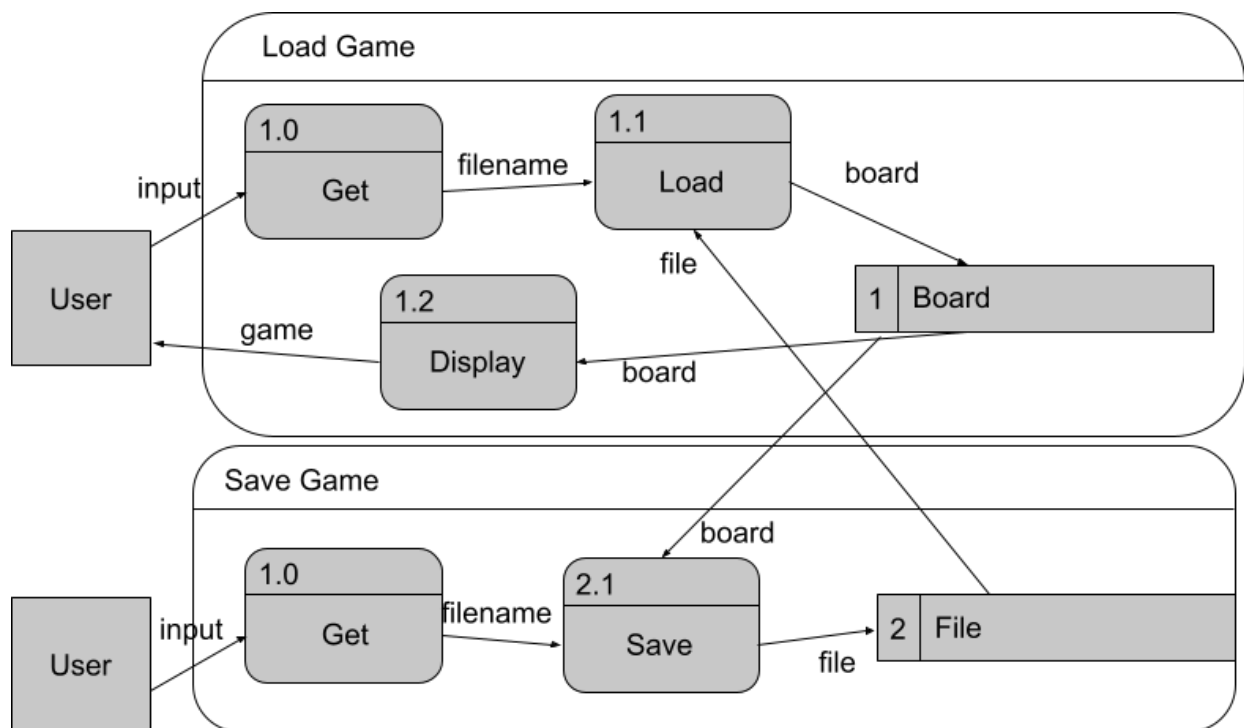


Structure Chart

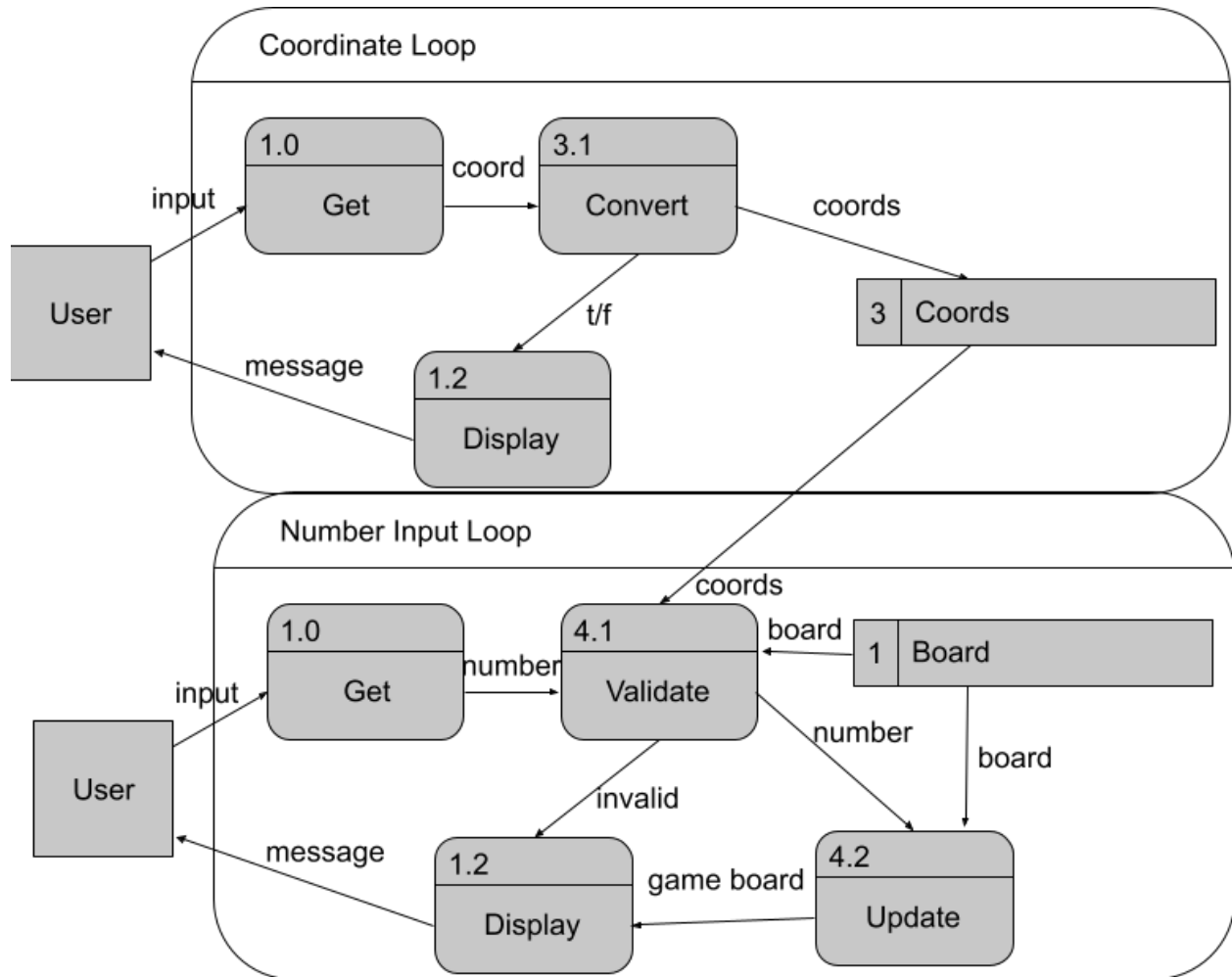


DFD

Save/Load Game



Game Loop



Pseudocode

convert_to_coordinate

```
convert_to_coordinate(input)
    # Split input into 2 parts
    TRY
        column ← input[0].to_upper()
        row ← input[1] - 1

    # Convert letter into number by subtracting ASCII code
    column ← ord(column) - 65
```

```

    # Assert coords are in range
    ASSERT  $0 \leq \text{column} \leq 8$ 
    ASSERT  $0 \leq \text{row} \leq 8$ 

    # Return coordinates
    RETURN (column, row)

# Error occurred along the way, return (-1, -1)
CATCH
    RETURN (-1, -1)

```

is_valid_choice

```

is_valid_choice(board, coord, num):
    row, col ← coord

    // --- Input validation ---
    IF num is not an integer:
        RETURN false
    IF num < 1 OR num > 9:
        RETURN false

    // --- Row check ---
    FOR c ← 0 ... 8:
        IF c ≠ col AND board[row][c] == num:
            RETURN false

    // --- Column check ---
    FOR r ← 0 ... 8:
        IF r ≠ row AND board[r][col] == num:
            RETURN false

    // --- 3x3 box check ---
    box_row_start ← (row / 3) * 3           // integer division
    box_col_start ← (col / 3) * 3

    FOR r ← box_row_start ... box_row_start + 2:
        FOR c ← box_col_start ... box_col_start + 2:
            IF (r, c) ≠ (row, col) AND board[r][c] == num:
                RETURN false

```

```
// --- All checks passed ---  
RETURN true
```

1. The prompt you provided to AI.

Generate the pseudocode for a function called `is_valid_choice` for a Sudoku program. It takes the current board (a 2d array), the coordinates of the square being modified (a tuple), and the user's input number. The coordinates have been checked to be valid but the user's input has not been checked. The function returns true if the number input would work for that square and false otherwise (e.g. invalid input, number breaks sudoku rules, etc.)

2. The results that AI generated.

```
FUNCTION is_valid_choice(board, coord, num):  
    row, col ← coord  
  
    // --- Input validation ---  
    IF num is not an integer:  
        RETURN false  
    IF num < 1 OR num > 9:  
        RETURN false  
  
    // --- Row check ---  
    FOR each c FROM 0 TO 8:  
        IF c ≠ col AND board[row][c] == num:  
            RETURN false  
  
    // --- Column check ---  
    FOR each r FROM 0 TO 8:  
        IF r ≠ row AND board[r][col] == num:  
            RETURN false  
  
    // --- 3x3 box check ---  
    box_row_start ← (row / 3) * 3      // integer division  
    box_col_start ← (col / 3) * 3  
  
    FOR r FROM box_row_start TO box_row_start + 2:  
        FOR c FROM box_col_start TO box_col_start + 2:  
            IF (r, c) ≠ (row, col) AND board[r][c] == num:  
                RETURN false  
  
    // --- All checks passed ---  
    RETURN true
```

3. Changes, improvements, and bug-fixes you made so the algorithm will work with your design.

The only changes I made were to the for loop 'syntax', making it match more closely to what we've used in class before.